

MetaboleAI — Spec di sviluppo: prodotti dinamici & wizard "Crea nuovo prodotto"

Handover tecnico per il team Sviluppo. Obiettivo: permettere di **creare un prodotto dal backoffice** (nome + menu propri + regole scelte) senza toccare il codice, e far girare **un agente AI per prodotto**.

Le **definizioni delle regole** non si duplicano qui: sono nel catalogo `Metabole_Regole_Motore_Catalogo.md` (Parte A). Questo documento copre **modello dati, wizard, API, agente per prodotto, integrazione pagina 16** e i **vincoli** da rispettare.

Regola ferrea (bloccante): **isolamento dei menu per prodotto** — vedi `STAT0.md` § Regole ferree e `Metabole_Motore_Personalizzazione.md` §0.

0. Requisito chiave — ZERO-REDEPLOY

Aggiungere o modificare un prodotto NON deve mai richiedere una ripubblicazione dell'app (né web, né nativa) né un deploy del backend. Il prodotto è **dato**, non codice.

- Il client (app web e nativa) **legge i prodotti dall'API a runtime** (`GET /products?active=1`), non da liste hardcodate. La pagina 16, i menu e le regole si popolano dai dati.
- Menu e ricette del prodotto arrivano dal server; l'app nativa resta invariata → **niente passaggio in App Store / Play Store** per un nuovo prodotto.
- Le regole sono data-driven (`ProductRule` + `config_param`): attivarle/parametrizzarle **non** è un deploy.
- Unica eccezione**: se un prodotto richiede un *tipo di schermata/interazione nuova* non ancora supportata dall'app, quella serve svilupparla una volta; poi tutti i prodotti che la usano restano data-driven. Obiettivo: le interazioni sono **generiche e configurabili**, così l'eccezione è rara.

1. Modello dati (delta)

- Product** : `id` , `name` , `slug` , `seasonal_tag` (nullable, es. "estate"), `objective` (`dimagrimento` | `mantenimento`), `status` (`bozza` | `in_review` | `attivo` | `archiviato`), `created_by` , timestamps.
- Menu** : **legato a product_id** (obbligatorio). `meal` (colazione/spuntino/pranzo/merenda/cena), `name` , `season` , `kcal` (interne), `status` . **Nessuna FK/riferimento a menu di altri prodotti**.
- Recipe** : `menu_id` , `steps` , `serving` (`caldo` | `freddo`), `status` . Appartiene al menu del suo prodotto.
- ProductRule** : `product_id` , `rule_code` (es. `L5` , `A2` ... dal catalogo), `enabled` (bool), `params` (JSON, es. soglie/pesi). Una riga per regola opzionale attivata. Le regole di sicurezza sono **implicite e sempre attive** (non servono righe, non disattivabili).
- RuleProposal** (coda "c'è un'altra regola?"): `product_id` , `text` , `proposed_by` , `status` (`pending` | `approvata` | `respinta`). Se approvata dal nutrizionista capo → diventa una nuova `rule_code` del catalogo.

Vincolo DB (S1): query e generazione menu **filtrano sempre per product_id** ; vietato qualsiasi join che unisca menu di prodotti diversi. A parità di piatti tra prodotti, le righe si **duplicano**.

2. Wizard "Crea nuovo prodotto" (backoffice)

Ruoli abilitati: nutrizionista / nutrizionista capo / admin. Passi:

- Anagrafica**: nome, tag stagionale (opz.), obiettivo (dimagrimento/mantenimento), descrizione.
- Menu**: inserimento menu **propri** per colazione/pranzo/cena (obbligatori) + spuntini/merende (opz.), ciascuno con ricette (steps + `caldo/freddo`). Salvataggio come `bozza` .
- Regole (consenso una a una)**: la UI presenta le regole **opzionali** del catalogo, una per schermata, con testo semplice; toggle `enabled` + eventuali `params` → crea/aggiorna `ProductRule` . Le di sicurezza sono mostrate come "sempre attive" (sola lettura).
- Proposta regola**: campo libero → `RuleProposal(pending)` .
- Revisione & attivazione**: `in_review` → il nutrizionista capo approva i menu (S6) → `attivo` . All'attivazione: **istanzia l'agente** del prodotto (vedi §4).

3. API (bozza)

POST	<code>/admin/products</code>	crea prodotto (bozza)
PATCH	<code>/admin/products/:id</code>	anagrafica/obiettivo/stato

POST	/admin/products/:id/menus	aggiunge un menu (+ ricette) al prodotto
PATCH	/admin/products/:id/rules	upsert ProductRule[] (enabled + params)
POST	/admin/products/:id/rule-proposals	coda "altra regola"
POST	/admin/products/:id/activate	in_review=attivo (dopo approvazione menu)
GET	/products?active=1&season=estate	lista prodotti per la pagina 16 (app)

Tutte le soglie/parametri restano coerenti con `config_param` (i `params` del prodotto sovrascrivono i default globali **solo per quel prodotto**).

4. Agente per prodotto

- Alla `activate`, si registra un'istanza di agente legata al `product_id`, configurata dalle `ProductRule` (stati A1–A6 attivi, pesi, obiettivo dimagrimento/mantenimento).
- L'agente ragiona **solo** sul catalogo del suo prodotto (già previsto in `Metabole_Agente_AI_Dieta.md` §8). Un'istanza di ragionamento per cliente.
- `objective=mantenimento` → efficacia neutra (nessun deficit); `dimagrimento` → spinta efficacia.

5. Integrazione pagina 16 (app cliente)

La schermata "Stile che preferisci" (pagina 16) non è più una lista statica: legge `GET /products?active=1` (con filtro stagionale se impostato). I prodotti esistenti (Mediterranea, Proteica, ecc.) diventano record `Product`; i due protocolli estate sono record come gli altri.

6. Vincoli & sicurezza (non negoziabili)

- **S1 isolamento menu** enforced a livello dati (nessun menu condiviso tra prodotti).
- **S2–S7** (allergie/patologie, sostituzione-o-blocco, porzioni standard, guardrail, solo menu approvati, riservatezza sanitaria) restano attive per **ogni** prodotto, sempre.
- Nessun segreto nel repo; audit su creazione/attivazione prodotto e su approvazioni.

7. Ordine consigliato di implementazione

1. `Product` + `Menu(product_id)` + `Recipe` + migrazione; migrare i regimi esistenti a `Product`.
2. `ProductRule` + mapping `rule_code` → comportamento (riusa i flag `config_param` già presenti).
3. Wizard backoffice (anagrafica → menu → regole → proposta → attivazione).
4. Agente per prodotto (istanza da `ProductRule`).
5. Pagina 16 dinamica (app legge i `Product` attivi).
6. `RuleProposal` + coda approvazione capo.

In una riga

Un `Product` possiede i **propri** menu (isolati) e un set di **regole scelte** (`ProductRule`); un **wizard** lo crea dal backoffice; alla attivazione parte **un agente AI per quel prodotto**; la **pagina 16** mostra i prodotti attivi. Le regole sono definite nel catalogo del motore, qui c'è il **come costruirlo**.